

LECTURE 5

Client-Server Chat Systems

Building Interactive Multi-User Applications

Today's Agenda

- 1 Chat System Architecture
- 2 Message Broadcasting
- 3 Private Messaging
- 4 Complete Chat Implementation
- 5 Advanced Patterns & Scaling
- 6 Modern Alternatives & Considerations

PART 1

Chat System Architecture

Designing multi-user communication systems

Requirements for a Chat Application

- Multiple concurrent users connecting simultaneously
- Message broadcast: one user sends, all others receive
- Private messaging: direct communication between two users
- User identification: distinguish messages by sender
- Graceful disconnect: handle users leaving cleanly
- Real-time: low latency message delivery

The Conscious Server



**Knows All Connected
Clients**

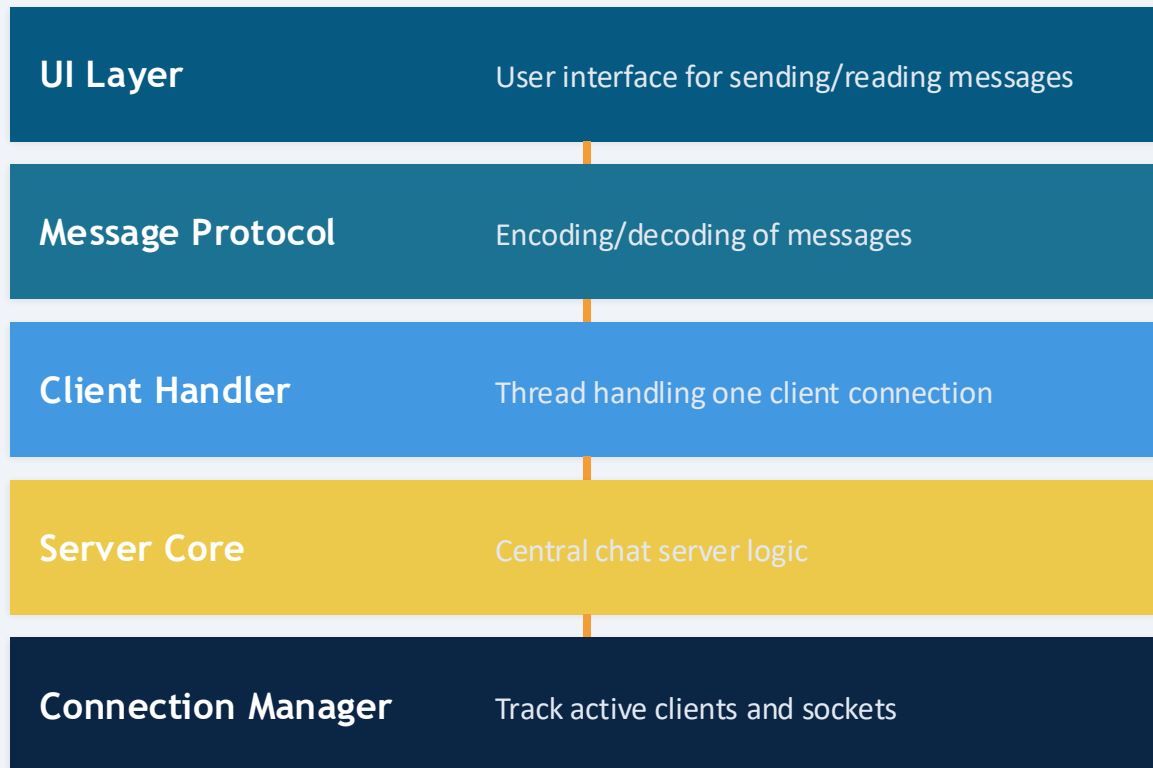
- Server maintains awareness of connected clients
- Tracks active connections in data structure
- Knows client names/IDs and their sockets
- Can route messages to specific clients
- Handles disconnection notifications
- Enables broadcast and private messaging

Conscious Server

A stateful server that maintains awareness of connected clients and their state, enabling intelligent message routing.

- Tracks active connections in data structure
- Knows client names/IDs and their sockets
- Can route messages to specific clients
- Handles disconnection notifications
- Contrast: stateless servers (like HTTP) don't remember context

Chat Application Architecture



Data Structures for Client Management

Vector<ClientHandler>

- Simple ordered list of clients
- Thread-safe with synchronization
- Easy iteration for broadcast
- Handles client removal
- Lower memory overhead

ConcurrentHashMap

- Maps username to ClientHandler
- O(1) lookup by name
- Built-in thread-safe iteration
- No synchronization needed
- Efficient for name-based lookup

Client Identification Strategies

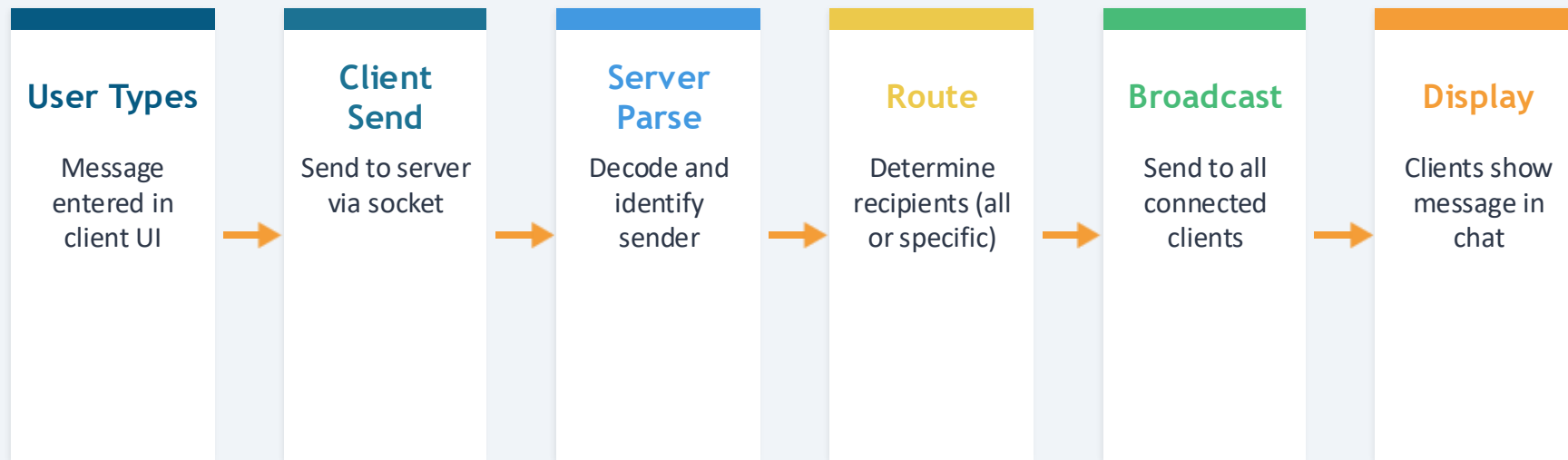
- Port-based ID: automatic, unique, no conflicts
- Username: user-friendly, requires management
- Session token: web-based systems, stateless
- MAC address: identifies physical machine
- IP address: problematic (dynamic, shared)
- Hybrid: auto-ID for protocol, username for users

PART 2

Message Broadcasting

Sending messages to multiple clients

Chat Message Flow: From User to All



Broadcast vs Private Messaging

Broadcast Messages

- Sent to all connected clients
- Visible to everyone
- Public chat room behavior
- One-to-many communication
- Can include sender or exclude

Private Messages

- Sent to specific recipient
- Visible to sender and recipient only
- Targeted communication
- One-to-one messaging
- Requires recipient lookup

Broadcasting to Connected Clients

- Server maintains list of active ClientHandlers
- For each connected client, send message
- Iterate through all ClientHandlers
- Call sendMessage(msg) on each handler
- Handle disconnected clients: remove from list if send fails
- Use synchronized block to prevent concurrent modification

Code: Broadcasting Messages

```
public class ChatServer {  
    private List<ClientHandler> clients =  
        Collections.synchronizedList(new ArrayList<>());  
  
    public void broadcast(String message,  
        ClientHandler sender) {  
        String fullMsg = sender.getUsername() +  
            ": " + message;  
  
        synchronized (clients) {  
            for (ClientHandler client : clients) {  
                if (client != sender) {  
                    try {  
                        client.sendMessage(fullMsg);  
                    } catch (Exception e) {  
                        clients.remove(client);  
                    }  
                }  
            }  
        }  
    }  
}
```

Synchronized prevents concurrent modification

Handling Disconnected Clients During Broadcast

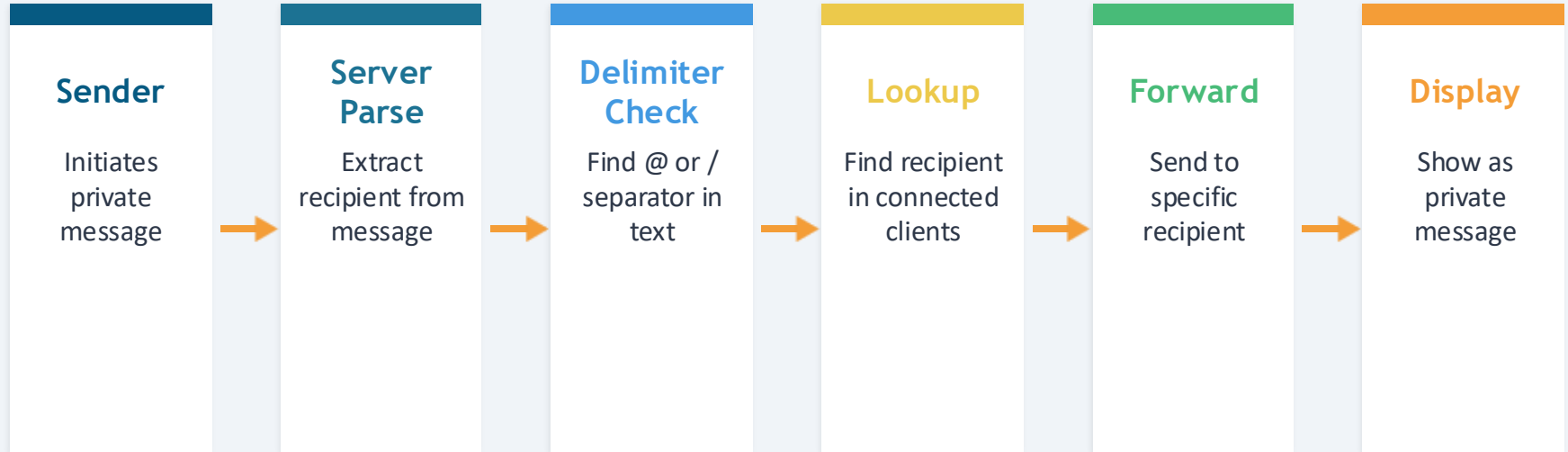
- Client may disconnect during broadcast iteration
- `sendMessage()` throws `IOException` if socket closed
- Catch exception and remove from clients list
- Must synchronize to prevent concurrent modification
- Alternative: use `Iterator.remove()` safely
- Notify remaining clients of departure

PART 3

Private Messaging

Targeted communication between users

Private Message Routing



Delimiter Strategies for Parsing

- @ prefix: '@username message' - Twitter-like
- / prefix: '/msg username message' - Command-like
- Space-delimited: 'username: message' - Simple but ambiguous
- Structured protocol: TYPE | RECIPIENT | MESSAGE
- JSON format: robust, handles edge cases
- Handle special characters: escape or encode

Code: Private Message Parsing and Routing

```
public void handleMessage(String msg,
    ClientHandler sender) {
    if (msg.startsWith("@")) {
        // Private message: @username message
        String[] parts = msg.substring(1)
            .split(" ", 2);
        if (parts.length == 2) {
            String recipient = parts[0];
            String text = parts[1];

            ClientHandler target = findClient(recipient);
            if (target != null) {
                target.sendMessage("Private from " +
                    sender.getUsername() + ": " + text);
            } else {
                sender.sendMessage("User not found");
            }
        }
    } else {
        // Broadcast message
        broadcast(msg, sender);
    }
}
```

Parse delimiter to route message appropriately

Code: Finding Connected Client by Username

```
private ClientHandler findClient(String username) {
    synchronized (clients) {
        for (ClientHandler ch : clients) {
            if (ch.getUsername()
                .equalsIgnoreCase(username)) {
                return ch;
            }
        }
    }
    return null;
}

// Better: use HashMap for O(1) lookup
private Map<String, ClientHandler> clientMap =
    new ConcurrentHashMap<>();

private ClientHandler findClient(String username) {
    return clientMap.get(username.toLowerCase());
}
```

HashMap provides better performance for lookups

PART 4

Complete Chat Implementation

Bringing it all together

Code: Chat Server Implementation

```
public class ChatServer {
    private ServerSocket serverSocket;
    private List<ClientHandler> clients;

    public ChatServer(int port) throws IOException {
        serverSocket = new ServerSocket(port);
        clients = Collections.synchronizedList(
            new ArrayList<>());
    }

    public void start() throws IOException {
        System.out.println("Chat server started");
        while (true) {
            Socket socket = serverSocket.accept();
            ClientHandler handler =
                new ClientHandler(this, socket);
            clients.add(handler);
            handler.start();
        }
    }

    public static void main(String[] args) {
        try {
            new ChatServer(5555).start();
        } catch (IOException e) { e.printStackTrace(); }
    }
}
```

Server accepts connections and creates handlers

Code: ClientHandler Thread

```
public class ClientHandler extends Thread {
    private ChatServer server;
    private Socket socket;
    private String username;
    private PrintWriter out;

    public ClientHandler(ChatServer s, Socket sock) {
        server = s;
        socket = sock;
    }

    public void run() {
        try {
            BufferedReader in = new BufferedReader(
                new
InputStreamReader(socket.getInputStream()));
            out = new PrintWriter(
                socket.getOutputStream(), true);

            username = in.readLine();

            server.broadcast(username + " joined", null);

            String msg;
            while ((msg = in.readLine()) != null) {
                server.handleMessage(msg, this);
            }
        } catch (IOException e) {}
        finally { cleanup(); }
    }

    public void sendMessage(String msg) throws
Exception {
        out.println(msg);
    }

    private void cleanup() {
        try { socket.close(); } catch (IOException e) {}
    }
}
```

Each handler manages one client

Chat System Features

Broadcasting

Send message to all connected users at once

Private Messages

Targeted communication between two users

User List

Display who is currently connected to chat

Join/Leave Notifications

Announce when users connect or disconnect

Message History

Optional: store chat logs for later review

Typing Indicators

Show when someone is typing a message

Building a Chat App: Step by Step



Simple Echo Server vs Conscious Chat Server

ECHO SERVER

- Receives message from one client
- Echoes back to same client
- No awareness of other clients
- No message routing
- Stateless



CONSCIOUS CHAT SERVER

- Receives message from one client
- Routes intelligently
- Knows all connected clients
- Can send to multiple or specific clients
- Stateful: remembers connections

PART 5

Advanced Patterns & Scaling

Building for scale and resilience

Hospital Chat System Requirements

15 min

SCENARIO

You're designing a chat system for a hospital where doctors need to communicate patient information. Messages must be private, encrypted, and logged for compliance. Some doctors are on-call and need message history when they come online.

THINK ABOUT

1

How would you modify the basic chat architecture?

2

What delimiter strategy handles special characters?

3

How do you deliver messages to offline users?

Think — Pair — Share

5 min

Scaling from 1,000 to 100,000,000 users

WhatsApp handles 100 billion messages per day. Our chat server uses one thread per client. At what scale does this architecture break? What would WhatsApp use instead?

1 THINK

Reflect individually for 1 minute

2 PAIR

Discuss with your neighbor for 2 minutes

3 SHARE

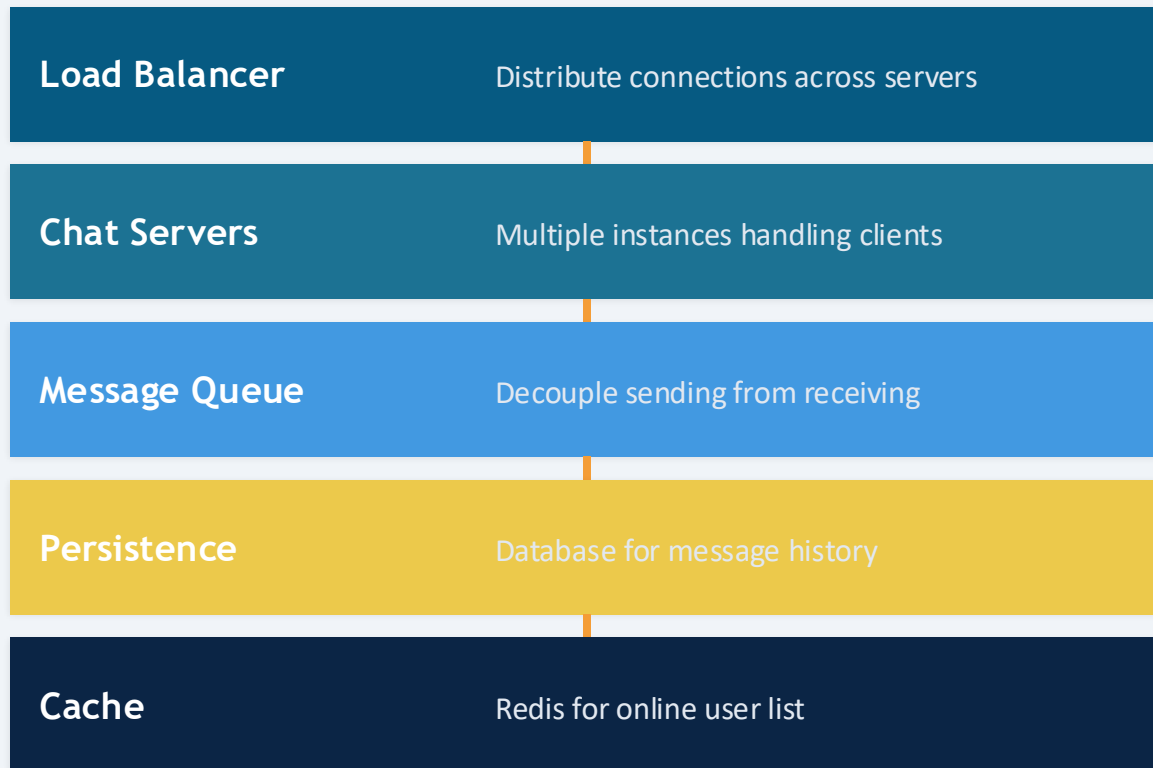
Share insights with the class

Hints: Consider OS thread limits | Memory per thread | Context switching overhead | Event-driven I/O | Message queues

Scaling Challenges with Thread-Per-Client

- Thread limits: OS has maximum thread count (~10k per server)
- Memory constraints: Each thread consumes 1-2 MB stack
- Context switching overhead: Too many threads = performance degradation
- Problem: 100 billion messages/day requires different approach
- Solution 1: Use thread pools instead of unbounded threads
- Solution 2: Use NIO (non-blocking I/O) for high concurrency

Scalable Chat Architecture



Message Queue Pattern for Scaling

- Decouple message receiving from delivery
- Sender writes message to queue (fast)
- Recipient reads from queue when ready
- Scales horizontally: add more servers
- Handles offline users: messages persist
- Examples: RabbitMQ, Kafka, Redis Streams

High-Performance Chat Architecture

- NIO (Non-Blocking I/O): Single thread handles thousands of clients
- Netty framework: Simplifies NIO programming
- Load balancing: Distribute across multiple servers
- Message persistence: Store in database for reliability
- Caching: Redis for online user list and session state
- Monitoring: Track latency, connections, message throughput

Technologies for Scaling Chat

NIO/Netty

Non-blocking I/O for thousands of concurrent connections

Load Balancer

Distribute traffic across multiple servers

Message Queue

RabbitMQ, Kafka for async message delivery

Database

Store message history and user data

Cache

Redis for fast user lookup and sessions

Monitoring

Track performance metrics and errors

PART 6

Modern Alternatives

Beyond traditional socket programming

WebSocket vs TCP Sockets

TCP Sockets

- Low-level network protocol
- Full control and flexibility
- Requires custom protocol design
- Works for any application
- Higher complexity

WebSocket

- High-level protocol over HTTP
- Built-in bidirectional communication
- Works in web browsers natively
- Better firewall/proxy support
- Simpler for web applications

Modern Chat Platforms & Services

- Firebase Realtime Database: Managed backend, auto-sync
- WebSockets: Native support in web browsers
- gRPC: Efficient binary protocol with code generation
- SignalR (.NET): Real-time bidirectional communication
- Socket.IO (Node.js): Fallbacks for browser compatibility
- Twillio/Sendbird: Hosted chat APIs

Manual Socket Chat vs Managed Platform

MANUAL SOCKETS

- Full control over implementation
- Custom protocol design
- Responsible for scaling
- Handle all failures
- High development effort



MANAGED PLATFORM

- Abstract away infrastructure
- Standard protocols/APIs
- Auto-scaling built-in
- Vendor handles reliability
- Focus on features

When to Build vs Buy Chat Systems

- Build if: Highly specialized requirements, cost sensitive, want full control
- Build if: Learning exercise, educational project, custom protocol needed
- Buy if: Need production reliability, rapid deployment, scaling concerns
- Buy if: Limited team, need vendor support, compliance requirements
- Hybrid: Use Netty/Socket.IO libraries, not raw sockets

Key Takeaways

- 1 Chat servers track connected clients and route messages intelligently
- 2 Broadcast sends to all clients, private messages to specific recipients
- 3 Use synchronized data structures to handle concurrent access safely
- 4 Parse message delimiters to distinguish message types and recipients
- 5 Thread-per-client works for small scale but doesn't scale to millions
- 6 NIO and message queues enable high-performance chat at scale

Building Your First Chat System

- Start simple: ServerSocket + threads accepting connections
- Add client tracking: List<ClientHandler> for connected clients
- Implement broadcast: Loop through clients, send message
- Add private messaging: Parse delimiter, lookup recipient
- Handle disconnects: Remove from list on IOException
- Test thoroughly: Multiple clients, rapid messaging, disconnections

Questions?

Next: Lecture 7 — JSON Data Interchange

CSCI 2450 | Computer Interface Programming | 2026

Real-World Enterprise Chat Stack

